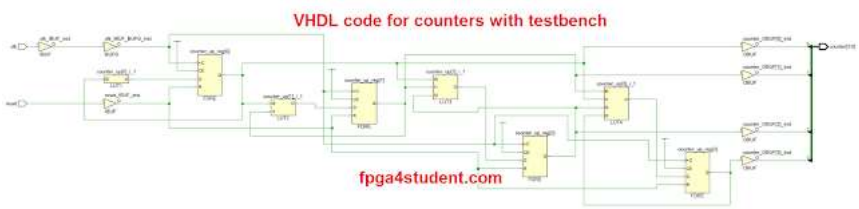
VHDL code for counters with testbench

Last time, several 4-bit counters including up counter, down counter and up-down counter are implemented in Verilog. Verilog code for the counters is presented.

In this VHDL project, the counters are implemented in VHDL. The testbench VHDL code for the counters is also presented together with the simulation waveform.



VHDL code for up counter:

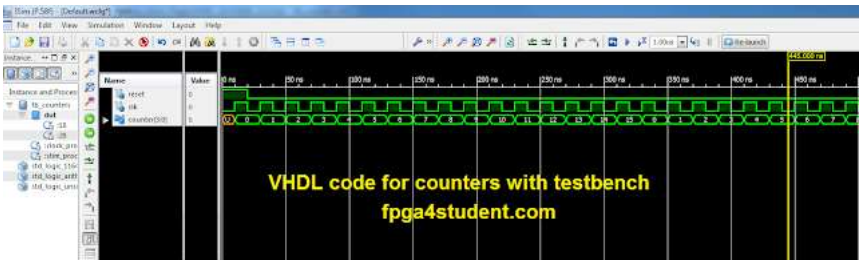
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- FPGA projects using Verilog code VHDL code
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for counters with testbench
-- VHDL project: VHDL code for up counter
entity UP_COUNTER is
    Port ( clk: in std_logic; -- clock input
           reset: in std_logic; -- reset input
           counter: out std_logic_vector(3 downto 0) -- output 4-bit counter
     );
end UP_COUNTER;

architecture Behavioral of UP_COUNTER is
signal counter_up: std_logic_vector(3 downto 0);
begin
-- up counter
process(clk)
begin
if(rising_edge(clk)) then
    if(reset='1') then
         counter_up <= x"0";
    else
        counter_up <= counter_up + x"1";
    end if;
 end if;
end process;
 counter <= counter_up;

end Behavioral;
```

Testbench VHDL code for the up-counter:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- FPGA projects using Verilog code VHDL code
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for counters with testbench
-- VHDL project: Testbench VHDL code for up counter
entity tb_counters is
end tb_counters;

architecture Behavioral of tb_counters is
```

```
component UP_COUNTER
    Port ( clk: in std_logic; -- clock input
           reset: in std_logic; -- reset input
           counter: out std_logic_vector(3 downto 0) -- output 4-bit counter
     );
end component;
signal reset,clk: std_logic;
signal counter:std_logic_vector(3 downto 0);

begin
dut: UP_COUNTER port map (clk => clk, reset=>reset, counter => counter);
   -- Clock process definitions
clock_process :process
begin
     clk <= '0';
     wait for 10 ns;
     clk <= '1';
     wait for 10 ns;
end process;


-- Stimulus process
stim_proc: process
begin
   -- hold reset state for 100 ns.
     reset <= '1';
   wait for 20 ns;
    reset <= '0';
   wait;
end process;
end Behavioral;
```

Simulation waveform:



VHDL code for down counter:

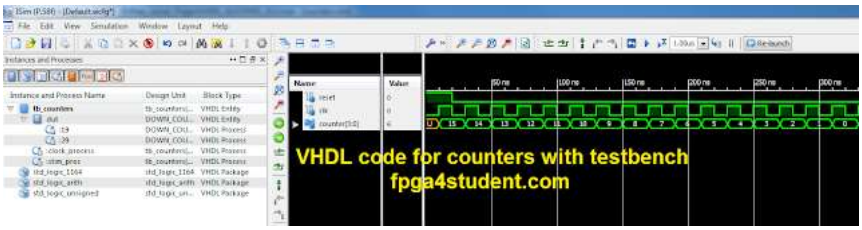
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- FPGA projects using Verilog code VHDL code
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for counters with testbench
-- VHDL project: VHDL code for down counter
entity DOWN_COUNTER is
    Port ( clk: in std_logic; -- clock input
           reset: in std_logic; -- reset input
           counter: out std_logic_vector(3 downto 0) -- output 4-bit counter
     );
end DOWN_COUNTER;

architecture Behavioral of DOWN_COUNTER is
signal counter_down: std_logic_vector(3 downto 0);
begin
-- down counter
process(clk)
```

```
begin
if(rising_edge(clk)) then
    if(reset='1') then
         counter_down <= x"F";
    else
        counter_down <= counter_down - x"1";
    end if;
 end if;
end process;
 counter <= counter_down;

end Behavioral;
```

Testbench VHDL code for the down counter:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- FPGA projects using Verilog code VHDL code
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for counters with testbench
-- VHDL project: Testbench VHDL code for down counter
entity tb_counters is
end tb_counters;

architecture Behavioral of tb_counters is

component DOWN_COUNTER
    Port ( clk: in std_logic; -- clock input
           reset: in std_logic; -- reset input
           counter: out std_logic_vector(3 downto 0) -- output 4-bit counter
     );
end component;
signal reset,clk: std_logic;
signal counter:std_logic_vector(3 downto 0);

begin
dut: DOWN_COUNTER port map (clk => clk, reset=>reset, counter => counter);
   -- Clock process definitions
clock_process :process
begin
     clk <= '0';
     wait for 10 ns;
     clk <= '1';
     wait for 10 ns;
end process;


-- Stimulus process
stim_proc: process
begin
   -- hold reset state for 100 ns.
     reset <= '1';
   wait for 20 ns;
    reset <= '0';
   wait;
end process;
end Behavioral;
```

Simulation waveform:



VHDL code for the up-down counter:

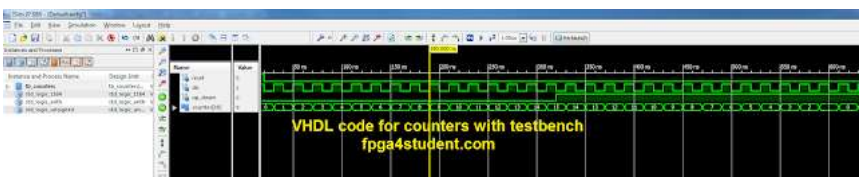
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- FPGA projects using Verilog code VHDL code
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for counters with testbench
-- VHDL project: VHDL code for up-down counter
entity UPDOWN_COUNTER is
    Port ( clk: in std_logic; -- clock input
           reset: in std_logic; -- reset input
     up_down: in std_logic; -- up or down
           counter: out std_logic_vector(3 downto 0) -- output 4-bit counter
     );
end UPDOWN_COUNTER;

architecture Behavioral of UPDOWN_COUNTER is
signal counter_updown: std_logic_vector(3 downto 0);
begin
-- down counter
process(clk)
begin
if(rising_edge(clk)) then
    if(reset='1') then
         counter_updown <= x"0";
    elsif(up_down='1') then
         counter_updown <= counter_updown - x"1"; -- count down
  else
   counter_updown <= counter_updown + x"1"; -- count up
    end if;
 end if;
end process;
 counter <= counter_updown;

end Behavioral;
```

Testbench VHDL code for the up-down counter:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- FPGA projects using Verilog code VHDL code
-- fpga4student.com: FPGA projects, Verilog projects, VHDL projects
-- VHDL project: VHDL code for counters with testbench
-- VHDL project: Testbench VHDL code for up-down counter
entity tb_counters is
end tb_counters;

architecture Behavioral of tb_counters is

component UPDOWN_COUNTER
    Port ( clk: in std_logic; -- clock input
           reset: in std_logic; -- reset input
     up_down: in std_logic;
           counter: out std_logic_vector(3 downto 0) -- output 4-bit counter
     );
end component;
```

```
signal reset,clk,up_down: std_logic;
signal counter:std_logic_vector(3 downto 0);

begin
dut: UPDOWN_COUNTER port map (clk => clk, reset=>reset, up_down => up_down, counte
   -- Clock process definitions
clock_process :process
begin
     clk <= '0';
     wait for 10 ns;
     clk <= '1';
     wait for 10 ns;
end process;


-- Stimulus process
stim_proc: process
begin
   -- hold reset state for 100 ns.
     reset <= '1';
   up_down <= '0';
    wait for 20 ns;
    reset <= '0';
  wait for 300 ns;
  up_down <= '1';
   wait;
end process;
end Behavioral;
```

Simulation waveform:



Recommended VHDL projects:

1. What is an FPGA? How VHDL works on FPGA
2. VHDL code for FIFO memory
3. VHDL code for FIR Filter
4. VHDL code for 8-bit Microcontroller
5. VHDL code for Matrix Multiplication
6. VHDL code for Switch Tail Ring Counter
7. VHDL code for digital alarm clock on FPGA
8. VHDL code for 8-bit Comparator
9. How to load a text file into FPGA using VHDL
10. VHDL code for D Flip Flop
11. VHDL code for Full Adder
12. PWM Generator in VHDL with Variable Duty Cycle
13. VHDL code for ALU
14. VHDL code for counters with testbench
15. VHDL code for 16-bit ALU
16. Shifter Design in VHDL
17. Non-linear Lookup Table Implementation in VHDL
18. Cryptographic Coprocessor Design in VHDL
19. Verilog vs VHDL: Explain by Examples
20. VHDL Code for Clock Divider on FPGA
21. How to generate a clock enable signal instead of creating another clock domain
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator

25. VHDL code for a single-port RAM
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
26. VHDL code for Car Parking System using FSM
27. VHDL coding vs Software Programming
28. VHDL code for MIPS Processor
29. VHDL code for Moore FSM Sequence Detector
30. VHDL code for Seven-Segment Display on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.